小米 HyperOS 预装组件数据采集与上报调查报告（公开版）

- 版本：v1.3.7
- 日期：2026-11-08
- 分析对象：一台 Xiaomi HyperOS 3.0 设备（具体型号、序列号、SIM 信息已隐去）
- 分析方式：静态反编译 + 运行时数据库取证 + 真机抓包 + 解密验证
- 说明：本文已移除全部个人隐私数据，仅保留技术事实与可复现方法

---

0. 摘要

在一台 HyperOS 3.0（Android 16）设备上，确认存在两套运营商终端管理（DM）客户端、一套电信自动注册组件、一套系统遥测组件、一套本地反诈数据库、一套 AI 通话防诈组件，以及系统包管理器的安装上报链路：

组件	包名	归属	服务器	状态
联通 DM	`com.mobiletools.systemhelper`	中国联通（Xiaomi 平台签名预装）	`dm.wo.com.cn:18080`（HTTP）	活跃，周期上报
移动 DM	`com.miui.dmregservice`	中国移动 / Richinfo	`a.fxltsbl.com:443`（HTTPS）+ CoAP 5683	按需唤醒，心跳正常
系统遥测	`com.miui.misightservice` + `/system_ext/bin/misight`(root)	Xiaomi	`event-log.cnbj1.mi-fds.com`	活跃
本地反诈库	`com.miui.securitycenter`	Xiaomi	未发现上传	本地写入

组件	包名	归属	服务器	状态
AI 通话防诈	`com.miui.guardprovider`	Xiaomi	`detection-aigc.engine.miui.com` 等	开关关闭，未启用
系统包管理器	`com.miui.packageinstaller`	Xiaomi	`api.installer.xiaomi.com` / `virus-engine.sec.miui.com`	活跃，安装/取消均上报
电信自动注册	`com.xiaomi.registration`	中国电信方向（Xiaomi 平台签名预装）	`zzhc.vnet.cn:443`（HTTPS）+ `typdm.tydevice.com:80`（HTTP）	活跃，已注册

主要结论：

1. `SystemHelper` **每 30 分钟采集**位置、基站、WiFi 并写入本地队列；**实际上报频率不是 30 分钟**：信号历史默认 **12 小时**一次、App 清单 **24 小时**一次，设备信息由开机/换卡/联网事件触发。上报内容包含设备标识、SIM 标识（部分打码）、经纬度（最多 6 位小数）、行政区、基站参数、WiFi 信息、存储信息与 App 使用清单。
2. 上报链路为 **明文 HTTP + AES-128-ECB**，且 **AES key = MD5(明文)**；`sign` 参数是 **RSA(1024 位公钥, MD5)**。由于 key 由明文派生，取得本地明文即可还原网络密文（已实测）。
3. `DMRegService` 采集 App 清单、完整 SIM 标识、基站/WiFi、按小时流量（含路由器 MAC 与公网 IP）、蓝牙/WiFi 列表，上报开关由云端配置控制。
4. 两套 DM 均存在 **无权限保护的导出组件**，本机任意应用可重定向上报地址或触发上报。
5. 截屏时间、安装记录、通话记录仅写入本地反诈库，**未发现上传路径**；AI 通话防诈当前关闭。
6. `com.xiaomi.registration` 是预装的电信终端注册组件，采集 MEID/IMEI/IMSI/ICCID/MAC/Cell ID 等标识并上报；本机在中国移动卡 + WiFi 条件下已完成注册，且存在无权限保护的导出组件。
7. `com.miui.packageinstaller` 在打开安装界面时即向 `api.installer.xiaomi.com` 发送包名/设备信息；**安装与取消都会**向 `/avl/upload` 上传安装元数据；云端还可按 `cInfo` 要求上传 APK 加密片段与 TLSH 模糊哈希。诈骗/风险判定由 GuardProvider 云查完成，命中后约 2 小时上报详细威胁报告。

---

1. 背景

部分小米/HyperOS 机型会随系统预装运营商定制的终端管理（DM）客户端。此类组件通常用于运营商侧的设备注册、网络优化与售后服务，但其数据采集范围、上报频率和加密方式缺少公开说明。

本报告基于一台已 root、已授权 ADB 的自用设备，尝试回答三个问题：

1. 设备上是否存在此类组件？
2. 它们采集什么、发往哪里、如何加密？
3. 用户能否关闭、关闭后是否仍然运行？

---

2. 方法

步骤	工具/手段
组件枚举	`pm list packages`、`dumpsys package`、`dumpsys alarm`、`dumpsys activity exit-info`
静态分析	`adb pull` 系统 APK → `jadx` 反编译 → 还原采集/加密/上报逻辑
证书分析	`openssl x509` / `openssl rsa`
数据库取证	直接读取 `/data/user/0/*/databases/*.db`、`/data/user_de/0/*/databases/*.db`
网络取证	`tcpdump -i any -w`、`ss -tnp`、`/proc/<pid>/cmdline` 做进程归属
解密验证	依据 `key = MD5(明文)` 重算密钥，与抓包密文逐字节比对，再用 `openssl enc -d -aes-128-ecb` 还原
触发实验	修改采集间隔（已备份并恢复）+ 广播 `BOOT_COMPLETED` 触发上报
安装器上报验证	打开/安装/取消 APK，抓 `logcat` 与 `tcpdump`，核对请求体

取证过程未修改业务数据；测试用配置已恢复原值。

---

3. 发现一：联通 DM（`com.mobiletools.systemhelper`）

3.1 基本信息

项	值
包名	`com.mobiletools.systemhelper`
APK	`/system/priv-app/SystemHelper/SystemHelper.apk`
版本	3.0.1-20260313（versionCode 34，targetSdk 34）
UID	10090
签名	Xiaomi MIUI 平台签名（`O=Xiaomi, OU=MIUI, CN=MIUI`）
内置证书	`assets/pub.cer`，自签名 `O=UNICOM`，RSA 1024 位，序列号 `528B9360`，2019-05-24 → 2119-04-30
证书指纹	`SHA256` `81:0C:01:22:BA:6A:F7:A4:B6:A5:92:1F:DB:BB:F9:19:7F:B2:E0:B2:E8:2D:E0:C5:AA:40:D2:8F:1B:7C:65:01`

3.2 采集内容

信号/位置（30 分钟一条，本地最多 100 条，约 50 小时）

```
Time / IMEI / ICCID / IMSI（部分打码）/ MNC / NetworkType
TAC / CID / PCI / ARFCN / Bands / RSRP / RSRQ / SINR
LONGITUDE / LATITUDE / District
BSSID / SSID
```

App 清单（RegType=A）

每个 App 包含：

```
appLabel / pkgName / runTime / launchCount
tx1 / rx1 / tx2 / rx2 / totalData1 / totalData2 / totalDataWiFi
```

过滤规则：排除 `com.android*`。

在测试设备上，该清单包含系统应用、第三方应用与 root/框架类工具（具体名单已隐去）。这意味着上报内容可用于判断设备是否被 root、安装了哪些工具。

3.3 上报协议

```
POST http://dm.wo.com.cn:18080/register?ver=5.0&model=<MODEL>&manuf=<MANUF>&sign=<RSA>
Content-Type: application/encrypted-json
Content-Length: <AES 密文长度>
User-Agent: Dalvik/2.1.0 (Linux; U; Android <VER>; <MODEL> ...)
```

加密流程：

```
key   = MD5(UTF8(plaintext))
body  = AES-128-ECB/PKCS5(plaintext, key)
sign  = Base64(RSA/ECB/PKCS1(pub.cer, key))
```

- `sign` 并非签名，而是 **RSA 包裹后的 AES key**；服务器用私钥解出 MD5 后解密 body。
- AES-ECB 无 IV，确定性加密：相同明文产生相同密文。
- 域名 CNAME：`63ce5fd2be92dcdf.icloudv6.com` → `223.111.128.82`。

3.4 队列与重试

- 本地库：`/data/user/0/com.mobiletools.systemhelper/databases/systemhelper`（表 `dmregister`）
- 队列封顶 11 条（按 `count-10` 删除旧记录后再插入）；HTTP 200/416 视为成功，其他状态（含 429）把明文写入队列
- 成功后会启动历史重试线程，弹出最新一条重发；该线程先删除后发送，失败即丢
- 实测队列规模约 10–11 条、约 187 KB，其中单条最大约 66–69 KB

3.5 触发条件

```
if (Settings.Secure.getInt("operator_service_improve_plan", 0) != 1) return false;
if (!networkConnected()) return false;
return (isWifi && isChinaMnc) || (!isRoaming && isUnicom);
```

> `isRoaming` 来自 `f1/a.java` 的 `ServiceState.getRoaming()`；`isChinaMnc` 检查 MNC 是否以 460 开头；`isUnicom` 只认

> 46001/46006/46009/46010。

3.5.1 采集频率 ≠ 上报频率

动作	周期	代码/默认值
信号/位置**采集**（写本地 `signal` 表）	**30 分钟**	`GetsignalInfoInterval = 1800000`
信号历史**上报**（R）	**默认 12 小时**	`signalInfoInterval = 43200000`；首次为随机 init(0–24h) + 12h
App 清单**上报**（A）	**24 小时**	`appInfoInterval = 86400000`
调度/设备信息（S）	**24 小时**	`scheduleInterval = 86400000`
设备信息（B/F）	**事件触发**	开机、换卡、联网、开机后 4 分钟
DMReg 心跳	24 小时	`heartbeat_interval = 86400000`
DMReg App 采集	7 天	`app_heartbeat_interval = 604800000`

> **30 分钟只是“采集到本地”，不是“每 30 分钟上报”。** 这也解释了为什么抓到的 R 包一次包含 100 条记录，而不是 1 条。

3.6 抓包/解密结果（脱敏）

包	密文字节	响应	结果
B（设备信息）	1,024	429	解密成功
R（信号历史）	68,784	429	**解密成功，100 条记录**
A（App 清单）	40,464	200	成功发出，本地无明文（不可从包解密）
A（App 清	40,464	**429**	**解密成功，169 个 App**

包	密文字节	响应	结果

单）

> A 包的 200 样本无法解密（成功不落库）；通过连续触发制造限流（429）后，同一明文会写入本地队列，即可完成“密文 → 明文”闭环。

R 包解密后的统计（不含具体值）：

- 记录数：100
- 覆盖约 2.3 天，30 分钟一条
- 基站：4 个 CID、3 个 PCI
- WiFi：2 个 SSID
- 经纬度：集中在约 100m × 20m 范围内
- 行政区：单一城区

A 包解密后的统计（不含具体值）：

- 记录数：169 个 App
- 字段：应用名、包名、使用时长、启动次数、移动/WiFi 收发
- 过滤：排除 `com.android*`
- 同一时段连续请求中，服务器返回 `200` 与 `429` 混合

3.7 无权限保护的导出组件

类型	组件	风险
service	`.service.RegisterHelperService`	AIDL 可改 serverAddress、改间隔、删配置、开关日志
activity	`.Activity.TestActivity`	打开即显示完整设备信息 JSON
receiver	`.control.OnBootReceiver`	任意应用可触发上报
receive	`.control.SIMStateReceiver`	任意应用可触发

类型	组件	风险
r		
receiver	`.control.ConnectStateReceiver`	任意应用可触发

`IRegisterHelperService` 的方法：

```
c(String addr);  // 修改上报服务器
a(int ms);       // 修改采集/上报间隔
e();             // 删除配置
d(boolean on);   // 开关日志
b();             // 读日志开关
```

实现中没有 `getCallingPackage()` 或权限校验，任意本机应用均可绑定调用。

---

4. 发现二：移动 DM（`com.miui.dmregservice`）

4.1 基本信息

项	值
包名	`com.miui.dmregservice`
APK	`/product/priv-app/RegService/RegService.apk`
版本	15.0.9
UID	1001（radio）
签名	Xiaomi MIUI 平台签名
开发方	Richinfo（`cn.richinfo.*`）

4.2 数据库

`/data/user_de/0/com.miui.dmregservice/databases/cmcc_dm.db`

表	内容
`app_info_*`	包名、UID、flags、启动次数、使用时长、移动/WiFi 收发
`table_name_network`	完整 IMSI、ICCID、cellid、lac、BSSID、SSID、band、RSRP、SINR、运营商
`table_of_time_related_traffic`	每小时流量、路由器 MAC、公网 IP
`table_name_battery_charge`	电量/温度/状态
`table_name_wifi_list` / `table_name_bluetooth`	WiFi/蓝牙扫描列表
`table_app_collect_list`	采集窗口与 SIM 标识

预装清单：`key_pre_install_info` 存有约 393 个预装 App 的元数据（名称、包名、版本、安装时间、签名 MD5、flags），尚未上报。

4.3 云端开关

开关	值
APP_REPORT_SWITCH	Y
key_network_report_switch	Y
KEY_TIME_RELATED_TRAFFIC_SWITCH	Y
key_wifi_list_report_switch	Y
key_bluetooth_report_switch	Y
key_heartbeat_switch	Y
key_pre_installed_report_switch	N
KEY_SANPSHOT_REPORT_SWITCH	N

开关	值
key_battery_charging_report_switch	N
key_5g_state_report_switch	N

周期：心跳 24 小时；App 采集 7 天；24 小时最多 5 次。

4.4 网络

- 主通道：`https://a.fxltsbl.com` → `117.134.71.39:443`（TLS）
- 备用：`https://b.fxltsbl.com`
- CoAP/LwM2M：`coap://m.fxltsbl.com:5683` → `117.134.71.81`（本机未观察到流量）
- 实名拍照：`https://auc.fxltsbl.com/AuthCenter/datareport?func=tsdk:photoReg`
- AES key 一部分硬编码于 `Constant.java`（`AES_KEY`），另一部分由 native `libaes-jni` 的 `keyFromJNI()` 提供

4.5 运行特征

- 未唤醒时静默（进程被系统回收、无闹钟）
- 收到 `cn.richinfo.dmyk.action.HEARTBEAT` 后立即启动，建立 2 条 TLS 连接，上传约 2.3 KB，收到约 0.3 KB
- `last_success_heartbeat_time` 随之更新，App 采集窗口同步延长

4.6 无权限保护的导出组件

类型	组件	风险
service	`cn.richinfo.dm.service.DMService`	任意应用可 startService 触发 DM 流程
receiver	`com.richinfo.mthttp.MTStaticBReceiver`	任意应用可发送 `alarm_heart`/`alarm_retry` 触发心跳
receiver	`.BootCompleteReceiver`	任意应用可触发

---

5. 发现三：系统遥测（MiSight）

项	值
守护进程	`/system_ext/bin/misight`（**uid=0 root**）
App	`com.miui.misightservice`（0.9.5，`/system_ext/app/MiSightService`）
本地库	`onetrack` / `one_track_cloud` / `onetrack_cache`
配置/策略	`https://event-log.cnbj1.mi-fds.com/event-log/...`
上传模式	`autoupload`，JobScheduler 周期 480 分钟
采集类别（云端配置）	application / audio_video / bluetooth / camera / cellular / charge / component / display / fingerprint / framework / gps / misight / network 等

取证时 `events` 表为空，未观察到大批量数据外发；主要行为是拉取配置与策略。

---

6. 发现四：本地反诈库（`com.miui.securitycenter`）

数据库：`/data/misc/user/0/securitycenter/fraud_detection_data.db`

表	说明
`screenshot`	仅毫秒时间戳（字段名拼写错误 `screenshort_time`）
`install_app`	包名、应用名、来源、版本、MD5、安装完成时间
`fraud_phone`	号码 MD5、归属地、运营商、时长
`call_forwarding`	呼叫转移状态
`voip_record` / `screen_share`	通话/投屏记录
`airplane_mode` / `fraud_url`	飞行模式/URL 记录

- 写入进程：`com.miui.securitycenter.remote`（uid 1000）
- 对外接口：`FraudDetectionProvider`（`exported=true`，signature 权限 `ACCESS_XSOF_FRAUD_DETECTION`）；

`MiFraudDetectionService` 为 `exported=false`，仅应用内/系统绑定

- 实测：按一次截屏，`screenshot` 表 +1 行，毫秒时间戳
- **未发现上传路径**：数据写入本地 SQLite，通过 ContentProvider / Binder 在本地暴露；未发现网络 DTO。`fraud-detection.engine.miui.com` 在 SecurityCenter 中仅用于 NFC 反诈配置 GET

---

7. 发现五：AI 通话防诈（`com.miui.guardprovider`）

> 一句话：这不是通用录屏工具。**录屏触发范围覆盖普通电话（未知号码）与微信/QQ 通话**；一旦触发，采集的是**整屏**，屏幕上是任何 App 都会进入画面。视频检测路径为**每轮录 3 秒 → 上传 → 未命中隔 2 秒再录**。当前开关为 0，未启用。

7.0 定位与边界

维度	范围
触发范围	**普通电话（未知号码）+ `com.tencent.mm`（微信）+ `com.tencent.mobileqq`（QQ）**
触发场景	语音/视频通话界面在前台（特定 Activity）
采集范围	**整屏镜像**（`AUTO_MIRROR` + 全屏尺寸）
产物	每段 3 秒的 `clip.mp4` → Base64 → 上传 `detection-aigc.engine.miui.com`
分析版本	预装 `2.5.6-20251106.0`；当前设备已更新到 `3.1.4-20260902.0`，3 秒循环与上传链路在 3.1.4 中仍存在
当前状态	开关 0、`RECORD_AUDIO` 未授权、无活跃 projection、无 clip 文件

7.1 录屏与上传链路

- `p079l1/d.java`：反射 `MediaProjectionManager.mService` → 直接 `createProjection`，绕过系统授权对话框

- `p079l1/a.java`：
- `createVirtualDisplay("ScreenRecorder", 整屏宽, 整屏高, dpi, 16, ...)`
- `16` = `VIRTUAL_DISPLAY_FLAG_AUTO_MIRROR`
- `MediaRecorder` 720×1280 / 24fps / 1Mbps；每次录制 3 秒（见 §7.1.1）
- 临时文件 `cache/clip_*`
- `AiGuardService.C(Path)`：
- `Files.readAllBytes` → Base64
- JSON：`{"streamName":"clip.mp4","fileBase64":"...","requestId":"..."}`
- 经 `com.miui.securitycenter.confidentialcomputing` 加密为 `EncryptedData`
- `POST https://detection-aigc.engine.miui.com/aiVideo/pcc/detection`
- 上传后 `Files.delete(path)`（成功失败都删）
- 音频文本走另一条：`POST /ai/audio/text/pcc/detection`

7.1.1 视频检测循环：录 3 秒 → 上传 → 未命中隔 2 秒 → 再录

`AiGuardService$p.A`（`COMMAND=2` 视频检测协程）是一个 `while (scope.isActive)` 循环：

```
while (scope.isActive) {
    recorder.c()                // MediaRecorder.start()
    delay(3.seconds)            // U3.c.o(3, SECONDS)
    val clip = recorder.d()     // MediaRecorder.stop()，返回 cache/clip_*
    if (clip != null) {
        val hit = C(clip)       // Base64 → 加密 → POST /aiVideo/pcc/detection
        if (hit) {              // checkRes == true
            u(scene)            // 弹警告
            stopSelf()
            break
        }
        Files.delete(clip)      // 未命中，删除
    }
    delay(2.seconds)            // U3.c.o(2, SECONDS)
}
recorder.b()                    // MediaRecorder.release()
```

- `3` 和 `2` 是方法内硬编码常量（`r6=3`、`r7=2`），不来自云端配置。
- 实际周期 = 3 秒录制 + 上传/等待云端响应 + 2 秒间隔，不是严格的“每 3 秒一次”。
- 命中后弹警告并 `stopSelf()`；未命中删除 `clip`，等 2 秒后继续。
- 若录制返回空（例如 `stop()` 失败），直接重试，不进入 2 秒等待。
- 单次服务生命周期由云端配置 `detectionServiceLifetimeThresholdInMinutes` 限制，默认 10 分钟；通话结束或服务停止时协程取消。
- 该循环只在 `COMMAND=2`（视频检测）路径；`COMMAND=1`（语音）走 ASR 文本检测，不使用这段录屏循环。
- `2.5.6-20251106.0` 与当前 `3.1.4-20260902.0` 均确认存在该循环；3.1.4 的对应方法为 `defpackage.x7.o`，JADX 同样报 `JadxOverflowException`，用 `--show-bad-code` 可还原。
- 3.1.4 中对应类：`defpackage.re3`（录制）、`defpackage.x7`（循环）、`defpackage.k43`（检测接口）、`defpackage.u23`（投屏）。

> 取证说明：JADX 默认模式在 `AiGuardService$p.A` 抛 `JadxOverflowException`，方法体被标记为 “not decompiled”。改用 `jadx --show-bad-code --comments-level debug` 重反编译后取得上述 smali 伪代码；`U3.c.o(n, SECONDS)` 与 `V3.W.b(...)`（Kotlin `delay`）已交叉确认。此前“录制类无可见调用点”的表述据此撤回。

因为 `AUTO_MIRROR` + 整屏尺寸，**录制范围是整屏，不是“只录微信”**；录制期间切到任何 App 都会被录进去。

7.2 触发链

```
SecurityCenter AiGuardSceneService
  ├── 监听 miui.media.AUDIO_VOIP_RECORD_STATE_CHANGED_ACTION
  ├── TelephonyManager 通话状态（普通电话）
  ├── 监听前台 Activity（IProcessObserver）
  |     ├── com.tencent.mm.plugin.voip.ui.VideoActivity   → 微信
  |     └── com.tencent.av.ui.AVActivity                  → QQ
  └── startForegroundService(com.miui.guardprovider/.aiguard.AiGuardService)
        COMMAND=1  PHONE_SCAM_SCENE=<scene>  CAPABILITIES=<bits>
        COMMAND=2  VIDEO_SCAM_SCENE=<scene>  CAPABILITIES=<bits>
```

- 微信/QQ 路径：`com.tencent.mm`、`com.tencent.mobileqq`，匹配对应 Activity
- **普通电话路径**：`TelephonyManager` 通话状态；`CALL_STATE=2` 且号码**不在联系人、也不在黄页**时，启动 `PHONE_SCAM_SCENE=2`

- 场景值：QQ `{3,1}`，微信 `{4,2}`
- `CAPABILITIES & 1` → 语音检测；`CAPABILITIES & 2` 且 `isUsingCamera` → 视频检测
- `AiGuardService.onStartCommand`：
- `COMMAND=3`：只同步 `ai_guard_function_switch`
- `COMMAND=1`：语音检测，要求 `RECORD_AUDIO`
- `COMMAND=2`：视频检测 → `P(scene)` → `p.A` 3 秒循环 → 每段 `C(Path)` 上传

7.3 三种“看屏幕”不是同一件事

功能	触发范围	采集范围	存什么
AI 防诈录屏	**普通电话 + 微信/QQ**	**整屏镜像**	`clip.mp4` 上传
截屏记录	任何 App（用户截屏）	仅事件	毫秒时间戳
屏幕共享记录	任何 App（除忽略名单）	仅包名	包名 + 起止 + 时长

7.4 局限性

1. **触发面有限**：覆盖普通电话（未知号码）与微信/QQ；Telegram、WhatsApp、Signal、钉钉、飞书等不在列表。
2. **依赖特定 Activity**：微信 `com.tencent.mm.plugin.voip.ui.VideoActivity`、QQ `com.tencent.av.ui.AVActivity`；版本更新、小窗、分屏、悬浮窗都可能导致匹配失败。
3. **视频依赖 `isUsingCamera`**：纯语音通话只走音频检测，不触发视频录制。
4. **`RECORD_AUDIO` 只约束语音路径**：`COMMAND=1` 在权限被撤销时打印 "permissions revoked" 并跳过；`COMMAND=2` 视频路径不检查该运行时权限，依赖 `MediaProjection` 与 `CAPTURE_VIDEO_OUTPUT`。
5. **依赖 `CAPABILITIES` 位**：bit1 语音、bit2 视频，由云端配置下发；位不满足就不启动。
6. **依赖云端开关**：`ai_guard_function_available` 与 `ai_guard_function_switch` 任一不满足都不会启动。
7. **3 秒/2 秒是硬编码节奏**：每轮固定录 3 秒，未命中后固定等 2 秒；上传与云端响应时间会拉长实际周期。视频路径已确认接线，调用点为 `AiGuardService$p.A`（见 §7.1.1）。

8. **全屏镜像的副作用**：一旦开始录，切到银行、密码、聊天等任何界面都会被录进 `clip.mp4`；不是“只录微信/QQ”。
9. **只在通话期间**：不是 7×24 录屏；通话结束即停止。
10. **自身录屏不进 `screen_share`**：`com.miui.guardprovider` 在忽略名单里，安全中心不会记录它自己的录屏。
11. **音频冲突/过热**：代码有音频冲突时停止的日志；过热中断未在代码中确认。
12. **不可见性**：录屏与上传没有用户可见确认；Android 14+ 可能显示隐私指示器，但授权对话框被绕过。

7.5 当前运行状态

检查项	结果
`ai_guard_function_switch`	0
`ai_guard_function_available`	false
`RECORD_AUDIO`（仅语音路径）	granted=false
活跃 MediaProjection	null
`cache/clip_*`	无
`ai_guard_daily_report.xml`	`<map />`（空）
`screen_share` / `voip_record`	0 行

> 当前设备为 `3.1.4-20260902.0`；上表已在 3.1.4 上复核；`available` 来自 SecurityCenter 的 `ai_guard_function_cloud_data.xml`。
>
> 视频路径不检查 `RECORD_AUDIO`；当前未启用的决定性条件是 `ai_guard_function_switch=0`、`available=false`、无活跃 `MediaProjection`、无 `clip` 文件。
>
> 结论：**能力存在，且视频路径已确认接线（3 秒一段、未命中隔 2 秒再录）；触发面窄、依赖多、当前未启用；一旦启用，录制范围是整屏，而不是只覆盖某个 App。**

---

8. 发现六：中国电信自动注册组件（`com.xiaomi.registration`）

> 一句话：这是小米预装的电信终端注册组件，平台签名、与电话应用共享 UID。它**不上传 App 清单**，采集 MEID/IMEI/IMSI/ICCID/MAC/Cell ID 等标识并上报；本机在中国移动卡 + WiFi 条件下已完成注册。

8.1 基本信息

项	值
包名 / APK	`com.xiaomi.registration` / `/product/priv-app/AutoRegistration/AutoRegistration.apk`
版本	15.1.22（versionCode 63，targetSdk 34）
签名 / UID	Xiaomi MIUI 平台签名 / **1001** **`android.uid.phone`**
进程 / 状态	`radio`；`RegistrationService` 活跃；12 小时闹钟已挂
APK SHA-256	`91f6324b07fe67cb753ac2875dd0c59f2b8cb9a43b97d92fb42b11b85f27c303`

Manifest 声明 16 个权限；因共享电话 UID，实际继承 `READ_SMS`、`READ_CALL_LOG`、`CALL_PHONE`、`CAMERA`、`WRITE_CONTACTS` 等。代码实际使用：

- `getImeiForSlot` / `getMeid`
- `getSubscriberId` / `getIccId`
- `WifiManager.getConnectionInfo`（MAC/BSSID）
- `CellInfo`（Cell ID/TAC）
- `SmsManager.sendDataMessage`（`10659401`）

未发现读取短信收件箱、通话记录、联系人或相机的代码。

8.2 三条上报路径

路径	地址	传输	本机
A	`https://zzhc.vnet.cn`	HTTPS；body 为	**是**

路径	地址	传输	本机
		Base64(JSON)	
B	`http://typdm.tydevice.com?jsonstr=...`	HTTP 明文；JSON 在 URL	否
C	短信 `10659401`	二进制短信，明文	否

路径 A 与 B 共用同一个选择条件，A 是 `z2 == false` 的分支：

```
z2 = (ro.radio.noril && ro.ril.oem.ctei 非空)
     || (Build.DEVICE in {ruan, koto, organ} && !Build.IS_CT_CUSTOMIZATION)


z2 == true  → B (typdm)
z2 == false → A (zzhc)
```

也就是说：**不是“无 RIL + CTEI”设备，也不是 `ruan/koto/organ` 三款非电信定制平板的设备，走 A。**

A 的实际发送还受 `p` 的运行时条件限制：

- 任一 SIM 就绪、非漫游、`device_provisioned=1`；
- SIM ICCID 变化、周期触发、系统版本变化、5G 开关变化或默认数据卡变化；
- 若当前是蜂窝网络且默认数据卡不是电信卡，则跳过；WiFi 下放行；
- 满足后由 `p` 调度 `RegistrationJobService`，走 `zzhc`。

路径 C 不是独立的顶层路径：它由 A 的 `p` 初始化（`p` 构造函数中创建 `j[]`）。选 B 时不会初始化 C；选 A 时 C 可与 A 的 HTTP 注册并存，但受电信卡、IMS 能力、在途注册 ID 限制。

路径 A：`i1/p.java` → `RegistrationJobService` → `i1/l.java`（按 `post_params.xml` 收集）→ `i1/m.java` case 0。响应 `resultCode == "0"` 视为成功。Header 写 `encrypted-json`，但代码只做 Base64，没有 AES/RSA。

路径 B：`j1/c.java` → `j1/b.java` → `i1/m.java` case 1。发送 `VER / CTEI / MAC / IP / UPLINKMAC / LINK / FWVER / DATE`。只在以下条件走：

```
boolean z2 =
    (e.c("ro.radio.noril") && !TextUtils.isEmpty(e.g("ro.ril.oem.ctei", "")))
    || d.d();
```

拆开看：

部分	代码	意思
左半	`e.c("ro.radio.noril")`	`SystemProperties.getBoolean("ro.radio.noril", false)`。表示设备没有常规蜂窝 radio（RIL）。
左半	`e.g("ro.ril.oem.ctei", "")`	`SystemProperties.get("ro.ril.oem.ctei", "")`。OEM 写入的 CTEI（China Telecom Equipment Identity，电信终端标识）；非空才满足。
左半整体	两者同时满足	没有常规蜂窝 radio、但 OEM 写了 CTEI 的设备。
右半	`d.d()`	`k1.d` 的静态方法，展开就是下面的“机型白名单 + 非电信定制版”。

`k1.d.d()` 展开：

```
return ("ruan".equals(Build.DEVICE)
        || "koto".equals(Build.DEVICE)
        || "organ".equals(Build.DEVICE))
       && !Build.IS_CT_CUSTOMIZATION;
```

右半拆项	意思
`Build.DEVICE in {ruan, koto, organ}`	设备代号是三款红米平板之一
`!Build.IS_CT_CUSTOMIZATION`	`Build.IS_CT_CUSTOMIZATION` 是 MIUI 的“电信定制版”标志；取反表示**非电信定制版**才命中

所以：

```
z2 = 左半（无 RIL + CTEI） || 右半（三款非定制平板）
```

右半就是 `k1.d.d()`，不是另一段独立逻辑。

代号对照：

代号	机型
ruan	Redmi Pad Pro 5G / POCO Pad 5G
koto	Redmi Pad 2 4G
organ	Redmi Pad 2 Pro 5G

它不是安全校验；返回 true 走 B（CTEI），false 走 A（Qualcomm/zzhc）。本机不在名单里，`z2 = false`，走路径 A。

路径 C：`i1/j.java` → `i1/q.java` → `i1/w.java`，发送

```
<a><b>型号</b><c>IMEI1</c><d>IMEI2</d><e>IMSI</e><f>系统版本</f><g>卡槽/网络</g></a>
```

要求 ICCID 以 `898603`/`898611`/`898612` 开头；本机是移动卡 `898602…`，未触发。

8.3 采集字段

路径 A（`post_params.xml`）：

```
REGVER / MEID / MODEL / SWVER
SIM1CDMAIMSI / SIM2CDMAIMSI / UETYPE
SIM1ICCID / SIM2ICCID
SIM1LTEIMSI / SIM2LTEIMSI
MACID / OSVER / REGDATE
IMEI1 / IMEI2 / DATASIM
RAM / ROM
SIM1CELLID / SIM2CELLID
FIVEGSW1 / FIVEGSW2
NWTYPE1 / NWTYPE2 / ACCESSTYPE / 5GRCSSW
SSW1 / SSW2
CARD1TYPE / CARD2TYPE
POWER
TAC1 / TAC2
```

路径 B 额外：`VER / CTEI / MAC / IP / UPLINKMAC / LINK / FWVER / DATE`。

路径 C：型号 / IMEI1 / IMEI2 / IMSI / 系统版本 / 卡槽与网络状态。

8.4 触发与本机状态

触发：开机、`PROVISION_COMPLETE`、`SIM_STATE_CHANGED`、`CONNECTIVITY_CHANGE/SERVICE_STATE`、两个 12 小时闹钟、`*#*#9999#*#*`；WiFi 下非电信卡也允许路径 A。

本机：

```
radio 4460 ... com.xiaomi.registration
com.xiaomi.registration/.autoregistration.RegistrationService
*walarm*:...SMS_AUTO_REGISTRATION_PERIOD_TRIGGER  (12h)
*walarm*:...AUTO_REGISTRATION_PERIOD_TRIGGER      (12h)
```

SharedPreferences：

```
<map>
  <string name="pref_meid">&lt;MEID&gt;</string>
  <string name="sim_iccid">&lt;ICCID，前 6 位 898602&gt;</string>
  <string name="register_regtime">2026-10-31 12:09:53</string>
  <string name="register_sw_version">&lt;系统版本&gt;</string>
  <int name="register_dds_slot" value="0" />
  <string name="autosms_provision">true</string>
</map>
```

`register_regtime` 有值，且同时有 `register_dds_slot` / `register_sw_version`，对应 `AutoRegReceiver` 的成功分支（路径 A）。`sim_iccid` 前 6 位 `898602` 是中国移动，所以本机是**移动卡 + WiFi** 完成电信方向注册。

8.5 问题与建议

#	问题
1	**跨运营商**：WiFi 下非电信卡也向 `zzhc.vnet.cn` 注册
2	**持久标识**：IMEI/MEID + IMSI/ICCID 可跨卡追踪设备
3	**位置推断**：Cell ID/TAC + BSSID 可推断粗略位置
4	**权限很大**：共享电话 UID，继承短信/通话记录/相机/联系人

#	问题

5 **入口开放**：`AutoRegReceiver` / `RegistrationService` 无权限保护

6 **明文 HTTP**：路径 B 在满足条件的机型上明文发送 MAC/IP/BSSID/CTEI

7 **Header 与载荷不符**：Header 写 `encrypted-json`，实际只是 Base64 编码

用户侧：不用电信卡时，可考虑 `adb shell pm disable-user --user 0 com.xiaomi.registration`，但可能影响电信 VoLTE/5G/IMS；或封堵 `zzhc.vnet.cn` / `typdm.tydevice.com`。

厂商/运营商侧：路径 B 改 HTTPS；给导出组件加签名权限或调用方校验；非电信卡明确提示或停止注册；提供用户开关与数据清单。

8.6 局限与证据

局限：

1. 未对 `zzhc.vnet.cn` 抓 TLS，POST body 为代码层推断。
2. “注册成功”由 `register_regtime` 与成功分支推断，未捕获服务器响应。
3. 路径 B/C 在本机未实际触发。
4. 基于单台设备、单版本 15.1.22；未评估禁用对电信 VoLTE/5G 的影响。

证据：

```
/product/priv-app/AutoRegistration/AutoRegistration.apk
/data/user_de/0/com.xiaomi.registration/shared_prefs/...
i1/{p,j,q,w,l,m}.java
j1/{b,c}.java
k1/{a,c,d}.java
com/xiaomi/registration/autoregistration/{AutoRegReceiver,RegistrationService,RegistrationJobService}.java
res/xml/post_params.xml
AndroidManifest.xml
```

---

9. 发现七：系统包管理器安装上报（`com.miui.packageinstaller`）

9.1 基本信息

项	值
包名	`com.miui.packageinstaller`
版本	5.5.7.0.0-20261012（versionCode 55700，targetSdk 34）
APK	`/data/app/.../com.miui.packageinstaller-.../base.apk`（系统应用更新）
关联组件	`com.miui.guardprovider` 3.1.4-20260902.0（病毒扫描/云查）
测试入口	文件管理器 `ACTION_VIEW` → `NewInstallerPrepareActivity`

边界：本节只覆盖 HyperOS 安装器 UI（打开 APK、点安装、点取消）。`adb install` / `pm install` 走 PackageInstaller 会话，是否触发同样的上报未在本节验证，两者行为可能不同。

9.2 三个时点的网络行为

- **打开安装界面（用户尚未确认）**：`/v3/cloud_config/query`、`/info/layout/v2` → `api.installer.xiaomi.com`；实测抓到。
- **点“安装”或“取消安装”**：`/avl/upload` → 同上；实测抓到。
- **安装前病毒扫描**：`/risk/scan`（3.1.4；2.5.6 为 `/virus/scan`）→ `virus-engine.sec.miui.com`；本次扫描超时。

- **命中威胁/黑灰名单后约 2 小时**：`/virus/localResult/report` → 同上；仅代码路径。
- **安装完成**：`content://com.xiaomi.security.xsof.FraudDetectionProvider/install_app` → 本地 SecurityCenter；代码/本地。

关键点：**打开安装界面就会发设备与包信息；点取消也会上传安装记录。**

9.3 `/avl/upload`：安装与取消都上报

成功安装与取消安装都会向：

```
https://api.installer.xiaomi.com/avl/upload
```

POST 一个 JSON。实测成功安装时（测试包为合成的 `com.example.pkgtest`）：

```
{
  "installCnt": "1",
  "packageName": "com.example.pkgtest",
  "name": "FuseFixer",
  "versionCode": "38",
  "versionName": "3574975",
  "apkHash": "<APK MD5>",
  "fingerprint": "<签名摘要>",
  "apkSize": "74594",
  "permission": "android.permission.QUERY_ALL_PACKAGES",
  "appSourcepackageName": "com.android.fileexplorer",
  "appSourceValue": "文件管理",
  "cdMd5": "<渠道摘要>",
  "installTime": "<时间戳>",
  "touchInstallCnt": "1"
}
```

取消安装时，字段基本一致，只有 `installCnt` 变为 `"0"`。服务器返回：

```
{"data":"succ","code":200,"msg":"succ"}
```

字段含义：

字段	含义
`packageName` / `name`	包名 / 应用名
`versionCode` / `versionName`	版本
`apkHash`	APK 文件 MD5
`fingerprint`	签名摘要

字段	含义
`apkSize`	APK 大小
`permission`	申请的全部权限（逗号连接）
`appSourcepackageName` / `appSourceValue`	安装来源包名 / 来源标签
`cdMd5`	渠道包摘要
`installTime`	安装/取消时间
`installCnt`	`1` 安装成功；`0` 取消
`touchInstallCnt`	点击安装计数
`apkPath`	调用方提供时附带（本次为空）

代码位置：

- 端点常量：`com/android/packageinstaller/I.java`（`f12442e = f12438a + "/avl/upload"`）
- 组装与发送：`com/miui/packageInstaller/util/a.java`（`InstallDataReporter`）
- 调用点：`InstallProgressParentActivity`（安装结果）、`p137t2/r.java` 与 `p116q2/c.java`（取消）

9.4 云端可控的 APK 片段与 TLSH

除安装元数据外，`/avl/upload` 还具备按云端指令取 APK 内容的能力：

1. 安装器向云端请求 per-app 配置，响应中的 `CloudParams.cInfo` 包含 `cFlag`、`cOffset`、`cSize`、`tFlag`。
2. `cFlag == true` 时，安装器从 APK 读取 `[cOffset, cOffset + cSize)` 区间。
3. 该区间用 `AES/CBC/PKCS5Padding` 加密：密钥为 `MD5(XiaoMiSafe.getKeyBytes())`，IV 为 `XiaoMiSafe.getKeyViBytes()`（均来自 `libxiaomiSafe.so`）。
4. 密文以 Base64 写入 `cToken`，同时上传 `cOffset`、`cSize`、`cMd5`（明文区间 MD5）。
5. `tFlag == true` 时，另外计算 APK 的 TLSH 模糊哈希，写入 `tInfo`。

代码位置：

- `com/miui/packageInstaller/model/CInfo.java`
- `com/miui/packageInstaller/util/a.java`（g 方法中的 `cToken`/`cMd5` 分支）
- `com/miui/packageInstaller/util/XiaoMiSafe.java`
- `F2/e.java`（`TlshManager`，`tInfo`）

需要强调：**这不是默认上传整包**。默认只上传元数据；只有云端返回 `cFlag/tFlag` 时才取片段/算 TLSH。若 `cSize` 被设为整个文件大小，等价于上传整包。该能力可用于完整性校验，但客观上也让服务器可以按需取走 APK 任意片段。

9.5 诈骗/风险应用：GuardProvider 云查与威胁上报

包管理器本身不判断“诈骗”，它把 APK 交给 `com.miui.guardprovider` 扫描。

云查请求

- 包管理器构造 `ScanRequestArgs.Builder(1)`（`SCAN_SCENE_PACKAGE_INSTALLER`），没有关闭云查；`ScanRequestArgs` 默认 `useCloudScan = true`。
- GuardProvider 向 `https://virus-engine.sec.miui.com/risk/scan`（3.1.4；2.5.6 为 `/virus/scan`）发送 `CloudScanApkInfo`：

字段	内容
`packageName`	包名
`versionCode`	版本号
`apkCertificateMD5`	证书 MD5
`apkCertificateSHA256`	证书 SHA-256
`mfFileSHA256`	manifest 文件 SHA-256

命中后的威胁上报

如果本地/云端结果为 `Threats` 或 `BlackGrays`，GuardProvider 会安排一个 `ReportWorker`：

- 约束：联网、初始延迟约 2 小时；
- 端点：`https://virus-engine.sec.miui.com/virus/localResult/report`；

- `ResultReportData` 字段：威胁等级、威胁名、恶意代码特征、包名、应用名、版本、证书 MD5/SHA-256、manifest SHA-256、APK MD5/SHA-256、MIUI 版本、Android 版本、设备名、UUID、`hitFiles`。

`hitFiles` 可能包含被命中文件的路径，属于需要重点脱敏的字段。

本地反诈写入

安装完成后，安装器还会向：

```
content://com.xiaomi.security.xsof.FraudDetectionProvider/install_app
```

插入 `pkg_name`、`app_name`、`install_finish_time`、`app_tag_id`、`version_code`。`app_tag_id` 来自云端三级分类；缺失时用 `-3/-2/-1`。这条记录写入本地 SecurityCenter 反诈库，未发现该 Provider 把它直接上传。

APK 清单中的反诈标签

安装器会读取 APK `<meta-data android:name="AntifraudLabel">`，并在 per-app 云控请求中以 `antifraudLabel` 字段发送。字段代码见 `com/miui/packageInstaller/C0647e.java`；读取代码见 `p095n2/c.java`。

9.6 实测与局限

测试方法：

- 设备：HyperOS 3.0.306.0.WOOCNXM / Android 16；
- 测试 APK：开源 FuseFixer 重打包，包名 `com.example.pkgtest`，debug v2 签名，无恶意代码；
- 操作：打开安装界面 → 安装一次；再次打开 → 取消一次；
- 取证：`logcat` + `tcpdump -i any`；
- 为拿到请求体，临时启用安装器的 `package_installer_log_on` 调试开关，测试后删除。

实测结果：

- `/avl/upload`：安装成功 `installCnt=1`、取消 `installCnt=0`，均返回 200；
- `/info/layout/v2`、`/v3/cloud_config/query`：打开安装界面即发出，参数含包名、应用名、安装来源、OAID、机型、区域、系统版本、网络类型等；
- pcap 中没有 `virus-engine.sec.miui.com` 的 DNS/SNI；
- 安装器日志出现 `virus scan timeout`，本次未完成 GuardProvider 云查；

- 本次未捕获到 per-app 云控请求的 URL；日志中的 `cp` 为 `catA=[null], sys=[true]`，未触发 `cToken`/`tInfo`；
- 未使用真实诈骗样本，`/risk/scan`（3.1.4；2.5.6 为 `/virus/scan`）与 `/virus/localResult/report` 为代码级证据。

因此，本节把结论分为两级：

结论	证据级别
安装/取消都会上传安装元数据	实测（logcat + pcap）
打开安装界面即上传包名/设备信息	实测（logcat）
云端可要求 APK 加密片段/TLSH	代码
GuardProvider 云查与威胁上报	代码；本次因扫描超时未跑通
本地反诈写入	代码；本地语义

9.7 问题与建议

#	问题
1	**取消也上传**：用户没有安装，安装记录仍被上报
2	**打开即上报**：尚未确认安装，包名/应用名/设备信息已发出
3	**字段过宽**：应用名、版本、签名、APK MD5、大小、全部权限、安装来源
4	**云端可控取片**：`cOffset/cSize` 由服务器决定，可读取 APK 任意区间并加密上传
5	**威胁报告含路径**：`hitFiles` 可能暴露用户文件路径
6	**无用户可见提示**：没有告知上传范围、云端取片或威胁报告

用户侧（可逆）

- 目前没有面向包管理器上报的公开开关；安装敏感 APK 时优先使用隔离环境/备用机；
- 不要在来源不明的 APK 上直接点“安装”或“取消”，因为两者都会触发 `/avl/upload`；

- 如使用 root 防火墙限制 `com.miui.packageinstaller` 联网，会影响云查/广告等在线功能，且未验证对安装成功率的影响，不能视为可靠缓解。

厂商侧

- 取消安装时停止 `/avl/upload`；
- 安装前以明确文案告知上传字段，并提供拒绝选项；
- 最小化设备标识与安装元数据；
- APK 片段/TLSH 上传改为单独授权，默认关闭；
- 威胁报告的 `hitFiles` 做路径脱敏；
- 提供上报记录查看与删除入口。

9.8 证据

代码路径：

```
com/android/packageinstaller/I.java
com/miui/packageInstaller/util/a.java
com/miui/packageInstaller/C0647e.java
com/miui/packageInstaller/c0.java
com/android/packageinstaller/C0609a.java
com/miui/guardprovider/sdk/ScanRequestArgs.java
F1/e.java                # 2.5.6 云查；3.1.4 对应 defpackage/bx.java
F1/t.java                # 2.5.6 上报；3.1.4 对应 defpackage/rd2.java
E1/A.java
com/miui/guardprovider/sdk/android/work/ReportWorker.java
```

日志与抓包：

- `logcat` 标签：`MIUIPI_NetUtil`、`MIUIPI_InstallDataReporter`、`MIUIPI_NewInstallerPrepareActivity`；
- 脱敏日志：`evidence/redacted/package_installer_upload.log`；
- 原始 pcap 仅在本地 `private/pcap/`，不入库；
- 关键类源码见 `evidence/decompiled/key/PackageInstaller/` 与 `evidence/decompiled/key/MIUIGuardProvider/`。

本次测试已移除 OAID、会话 ID、设备型号等标识；测试 APK 已卸载，调试开关已删除。

---

10. 安全与合规问题

1. **明文 HTTP 传输元数据**：请求走 `http://`，URL、机型、厂商、时间、大小、`sign` 可被中间人观察。
2. **弱加密设计**：AES-128-ECB、无 IV、`key = MD5(明文)`，确定性加密；相同明文产生相同密文。
3. **1024 位 RSA + 100 年有效期**：自签名证书，2019–2119；结构正常（popcount 49.7%，无小因子、无共享素数、无近似素数），但 1024 位已低于现代标准。
4. **无权限保护的导出组件**：任意本机应用可修改 `SystemHelper` 的上报服务器、触发上报、清空配置；可启动移动 DM 服务、触发心跳。
5. **可观测性缺失**：两套 DM 均无用户可见开关/日志；对联通 DM 而言，`operator_service_improve_plan` 是唯一总闸；移动 DM 由云端开关单独控制。
6. **加密可逆性**：由于 key 由明文派生，任何能获取本地明文（root、备份、队列）的一方都可解密网络流量。
7. **电信自动注册组件**：路径 B 使用明文 HTTP（MAC/IP/BSSID/CTEI）；`AutoRegReceiver` / `RegistrationService` 导出且无权限保护；`zzhc` 载荷仅为 Base64。
8. **包管理器取消也上报**：用户未安装，`/avl/upload` 仍上传包名、应用名、版本、签名、APK MD5、大小、权限与来源。
9. **打开安装界面即上报**：尚未确认安装，安装器已向 `api.installer.xiaomi.com` 发送包名、应用名、OAID、机型、区域等信息。
10. **云端可控取片**：`cInfo.c0ffset/cSize` 由服务器决定，可读取 APK 任意区间并 AES-CBC 加密上传；`tFlag` 触发 TLSH 模糊哈希。
11. **威胁报告含路径**：GuardProvider 命中威胁后上报的 `hitFiles` 可能包含用户文件路径。

---

11. 隐私影响（定性）

可被还原的画像维度：

维度	内容
设备身份	设备标识、SIM 标识、机型、系统、存储
位置	经纬度、行政区、基站、WiFi、路由器公网 IP

维度	内容
行为	每个 App 的使用时长、启动次数、流量
安装	安装来源、版本、安装时间、部分签名；包管理器另上报应用名、APK MD5/大小/权限、安装或取消
截屏	毫秒时间戳（仅本地）
Root/框架	可识别 root 与框架类工具

两套 App 清单合并后，可判断设备的 root 状态、常用应用、作息与位置轨迹。

---

12. 建议

12.1 用户侧（可逆）

```
# 1) 关闭运营商服务改进计划（总闸）
adb shell settings put secure operator_service_improve_plan 0

# 2) 清空待传队列
adb shell su -c 'rm -f /data/user/0/com.mobiletools.systemhelper/databases/systemhelper*'
adb shell su -c 'rm -f /data/user/0/com.mobiletools.systemhelper/databases/signalhelper*'
adb shell su -c 'rm -f /data/user_de/0/com.miui.dmregservice/databases/cmcc_dm.db*'

# 3) 停用组件（会影响运营商相关功能，建议先做 1+2）
adb shell su -c 'pm disable-user --user 0 com.mobiletools.systemhelper'
adb shell su -c 'pm disable-user --user 0 com.miui.dmregservice'

# 4) 域名封堵：dm.wo.com.cn / a.fxltsbl.com / m.fxltsbl.com / auc.fxltsbl.com

# 5) 包管理器目前无公开的上报开关；安装敏感 APK 时优先使用隔离环境/备用机。
#    限制 com.miui.packageinstaller 联网会影响云查/广告，且未验证安装成功率，不能视为可靠缓解。
```

12.2 厂商/运营商侧

- DM 通道改为 HTTPS，或至少使用 AEAD（AES-GCM）与随机 IV；
- 停止使用 `key = MD5(明文)`，改为标准密钥协商；
- 升级到 ≥2048 位 RSA 并缩短证书有效期；
- 为导出组件增加 signature 权限与调用方校验；
- 提供用户可见的开关、数据清单与删除入口；
- 明确告知预装 DM 客户端的存在与采集范围。
- 包管理器取消安装时停止 `/avl/upload`；安装前明确告知字段并提供拒绝选项；
- 默认关闭 APK 片段/TLSH 上传，改为单独授权；
- 威胁报告的 `hitFiles` 做路径脱敏，并提供上报记录查看/删除入口。

---

13. 附录：IOC

域名 / IP

```
dm.wo.com.cn:18080              → 223.111.128.82 (CNAME 63ce5fd2be92dcdf.icloudv6.com)
a.fxltsbl.com:443               → 117.134.71.39
b.fxltsbl.com:443
m.fxltsbl.com:5683              → 117.134.71.81 (CoAP)
auc.fxltsbl.com:443
event-log.cnbj1.mi-fds.com:443
zzhc.vnet.cn:443
typdm.tydevice.com:80
api.installer.xiaomi.com:443    → 安装器云控 / 布局 / avl 上报
virus-engine.sec.miui.com:443   → GuardProvider 云查 / 威胁上报
```

包名 / 路径

```
com.mobiletools.systemhelper    /system/priv-app/SystemHelper/SystemHelper.apk
com.miui.dmregservice           /product/priv-app/RegService/RegService.apk
com.miui.misightservice         /system_ext/app/MiSightService
com.miui.securitycenter         /data/misc/user/0/securitycenter/fraud_detection_data.db
```

```
com.miui.guardprovider          /product/app/MIUIGuardProvider
com.xiaomi.registration         /product/priv-app/AutoRegistration/AutoRegistration.apk
com.miui.packageinstaller       /data/app/.../com.miui.packageinstaller-.../base.apk（系统应用更新）
```

核查命令

```
adb shell pm path com.mobiletools.systemhelper
adb shell dumpsys package com.mobiletools.systemhelper | grep -E "versionName|appId|codePath"
adb shell settings get secure operator_service_improve_plan
adb shell su -c 'sqlite3 /data/user/0/com.mobiletools.systemhelper/databases/systemhelper \
  "select ID, length(jsonString) from dmregister"'
adb shell ss -tnp | grep -E "18080|:443"
adb shell dumpsys package com.miui.packageinstaller | grep -E "versionName|versionCode"
adb logcat -d | grep -E "MIUIPI_NetUtil|MIUIPI_InstallDataReporter|virus scan timeout"
```

13.1 证据包

完整证据随报告仓库提供：

```
evidence/
├── MANIFEST.md                 # 每个文件的 SHA-256
├── SHA256SUMS                  # 可直接 sha256sum -c 校验
├── apk/                        # 6 个 APK（本地保留，不入库）
├── decompiled/
│   ├── key/                     # 66 个关键类源码（入库）
│   └── full/                    # 完整 jadx 源码树（本地保留）
├── certificates/               # pub.cer / pub.pem / openssl 文本
├── databases/                  # 仅 schema 与行数
└── redacted/                   # 脱敏样本与日志
```

> 原始 pcap 仅本地保留（`private/pcap/`），不入库。

校验：

```
sha256sum -c evidence/SHA256SUMS
```

> 原始数据库、解密后的 CSV/JSON、含隐私的内部报告均在 `private/`，不随公开仓库分发。

---

14. 限制与声明

- 本报告基于单台设备、单次取证，不代表所有小米机型/系统版本。
- 反编译代码可能因混淆存在偏差；关键结论尽量通过运行时数据与抓包交叉验证，无法实测的已在正文标注。
- 取证过程未修改业务逻辑；测试用配置已恢复原值。
- 本文已移除设备标识、SIM 标识、位置、WiFi、App 清单等个人隐私信息，仅保留技术事实。
- 文中所述组件均为系统预装、Xiaomi 平台签名，不代表其具有恶意意图；报告关注的是采集范围、加密方式与用户可控性。
- 包管理器部分基于单台设备、单次安装/取消测试；未使用真实诈骗样本，`/risk/scan`（3.1.4；2.5.6 为 `/virus/scan`）与 `/virus/localResult/report` 未在本次跑通；per-app 云控请求未捕获，`cToken/tInfo` 未触发；`adb install` / `pm install` 路径未覆盖。